VulCrypt: A Cryptographic-API Misuse Detector

Software Project Lab 3 Technical Report

Submitted by
Abir Ahmed
BSSE 1229
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Adwait Nadkarni
Associate Professor,
Department of Computer Science,
William & Mary
Williamsburg, VA, USA

&

Mridha Md Nafis Fuad
Lecturer
IIT, University of Dhaka
Dhaka, Bangladesh

Mentored by
Md. Akram Khan
Ph.D. Student
William & Mary
Williamsburg, VA, USA

_____

Supervisor's Signature
Mridha Md Nafis Fuad
Lecturer
IIT, University of Dhaka

Date: 01 December 2024

IIT
University of Dhaka

Letter of Transmittal

01 December 2024

BSSE 4th Year Exam Committee
Institute of Information Technology,
University of Dhaka

Subject: Technical report on “VulCrypt: A Cryptographic-API Misuse Detector”.

Sir,

With due respect, I am pleased to submit the midterm report of Software Project Lab-III on “VulCrypt: A Cryptographic-API Misuse Detector”. I have tried my best to deliver a good report. However, it might lack perfection.

So, I, therefore, hope that you would be kind enough to accept my report and oblige thereby.

Sincerely yours,
Abir Ahmed
BSSE 1229
IIT, University of Dhaka.

Acknowledgement

Firstly, I would like to thank the Almighty Allah for helping me complete my 4th Year Final Project Software Requirements and Specifications.

I am grateful to the Institute of Information Technology, University of Dhaka for giving me such a tremendous opportunity to work on “VulCrypt: A Cryptographic-API Misuse Detector” at William & Mary, under the supervision of Adwait Nadkarni, Associate Professor, Department of Computer Science, William & Marry, Williamsburg, VA.

I would like to convey my tremendous gratitude to the BSSE 4th year exam committee, my internal supervisor, Mridha Md Nafis Fuad, Lecturer, Institute of Information Technology, University of Dhaka, and my mentor for providing me guidelines about how I can complete this project. They helped me a lot by sharing their valuable knowledge with me. My parents have been a great mental support for me. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

This document contains the software requirements and specifications, architectural design, component design, user interface design, implementation details, testing, and user manual of “VulCrypt: A Cryptographic-API Misuse Detector”. This tool can be used to detect cryptographic vulnerabilities harnessing the power of Machine Learning. It identifies vulnerabilities by slicing code, extracting cryptographic API usage, and generating detailed visualizations of misuse patterns. Leveraging external tools like MASC for mutants generation and JADX for decompilation, VulCrypt provides developers with actionable insights to enhance application security. Our proposed tool is a prototype for the entire research.

Table of Contents

Chapter 1: Introduction **6**
1.1 Purpose 6
1.2 Scope 6
1.3 Assumptions 7
Chapter 2: Project Description **7**
2.1 User Scenario 7
2.2 Motivation 8
Chapter 3: Quality Function Deployment(QFD) **9**
3.1 Normal Requirements: 9
3.2 Expected Requirements: 9
3.3 Exciting Requirements: 9
Chapter 4:Timeline **10**
Chapter 5: Use Case Diagram **11**
Level 0: VulCrypt 11
Level 1: Modules of VulCrypt 12
Chapter 6: Class Based Diagram **14**
6.1 Analysis Classes 14
6.2 Class Cards 14
6.3 Class Responsibility and Collaboration Diagram 18
Chapter 7: Architectural Design **19**
7.1 Architecture Overview 19
7.2 Architecture Context Diagram 19
7.3 Archetype Definition 20
7.4 Refining Archetypes into Components 21
Chapter 8: Primary Test Plan **21**
8.1 High-Level Description of Testing Goals 21
8.2 Test Cases 22
Chapter 9: Conclusion **26**
Reference **27**

Chapter 1: Introduction

The rise in security vulnerabilities due to the improper use of cryptographic APIs in software has become a significant concern for developers and security analysts alike. Current static analysis tools and machine learning (ML) based detectors provide varying degrees of success in identifying these vulnerabilities. However, a robust and scalable approach that integrates the strengths of ML and static analysis is still needed.

This project aims to evaluate and enhance the effectiveness of ML-based crypto API detectors. By leveraging the “Mutation Analysis for evaluating Static Crypto-API misuse detectors **(MASC)**” to generate mutants and training ML models on these mutations, the project will identify gaps in existing tools and develop a comprehensive solution that bridges ML-based and static analysis-based detection approaches.

1.1 Purpose

The purposes of this document are:

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

1.2 Scope

The scope of the project is given below:

- It will be a Web Based Application to detect cryptographic vulnerabilities.
- The system will train from a Transformer such as codet5, codegpt, etc.
- The system will use a portion of MASC-generated mutants to train the model.

1.3 Assumptions

The assumptions of the project are:

- The system will have a stable internet connection.
- The input codebase should not contain any god class (more than 500 lines in a single java file)
- Tools perform better when more data is provided for training.

Chapter 2: Project Description

2.1 User Scenario

The goal of this project is to show the users the cryptographic vulnerabilities within a Java codebase they provided using a machine learning based tool which is our proposed analysis. Not only do the Crypto Vulnerabilities themselves have to be shown but also their locations have to be provided.

A web application where users will be able to upload the codebase in zipped format or an Android application as APK and the application will analyze it through a machine learning model, then display the combined result and the location of the vulnerabilities.

On the home page, users will be able to upload an APK or zipped code, then they have to select the analysis mode which is Crypto Analysis or just only Data Flow Analysis.

Users can select one, multiple, or all of them allowing them to view the result from the particular software(s). After selecting from the list, users can generate results and based on their choices, the result summary will be provided by the software. One can further inspect details.

This is a helpful tool for developers as they can find where their vulnerabilities are and what it is. Also, since the application shows the data flow of the code, the problems that are frequently missed in-app life cycles can be fixed.

2.2 Motivation

Cryptographic APIs are essential tools that enable developers to offer a standardized approach to implementing complex cryptographic functions, such as encryption and digital signing, without requiring extensive knowledge of cryptography. Its misuse is a widespread problem in software development, especially among developers who may not have specialized knowledge of cryptography.

Insecure implementations, such as using weak encryption algorithms or hardcoded keys, can lead to security breaches and data loss. By leveraging secure implementations of cryptographic algorithms, developers can concentrate on building their applications while ensuring the protection of sensitive data and establishing secure communication channels. Consequently, crypto-APIs play a crucial role in developing secure and trustworthy software.

Existing tools are often based on rigid, rule-based systems that fail to detect many forms of misuse, especially when the code doesn’t exactly match predefined patterns. By using machine learning, our detector aims to offer a more flexible and adaptive solution that can learn from diverse examples of code misuse and identify vulnerabilities more accurately.

Chapter 3: Quality Function Deployment(QFD)

Quality Function Deployment is a structured approach to translating customer requirements (in this case, project motivations and user stories) into detailed product or project requirements. Here are the normal, expected, and exciting requirements:

3.1 Normal Requirements:

1. Upload zipped Java codebase to the web application to check for possible vulnerabilities within it.
2. Use Machine Learning techniques to detect vulnerabilities.
3. Display the list of the found vulnerabilities.
4. The tool should have reasonable processing times for codebase analysis, ensuring minimal impact on user experience.

3.2 Expected Requirements:

1. Show the file’s location of the found vulnerabilities.
2. VulCrypt's machine learning classifiers should demonstrate a high level of accuracy in identifying crypto vulnerabilities.
3. Handle software systems of various sizes and complexities.

3.3 Exciting Requirements:

1. Support APK decompilation to ensure the analysis of Android applications.
2. Support analysis of multiple codebases(zip or apk) at one go.

Chapter 4:Timeline

An 18-week timeline has been planned for this project.

VulCrypt Timeline																			
Task Name	**Duration (In Weeks)**	Week-1	Week-2	Week-3	Week-4	Week-5	Week-6	Week-7	Week-8	Week-9	Week-10	Week-11	Week-12	Week-13	Week-14	Week-15	Week-16	Week-17	Week-18
Knowledge Gather & Proposal Preparation	2																		
SRS Documentation	4																		
Project Design	4																		
Project Development	10																		
Project Testing	6																		
Report Submission	2																		

Figure 1:Timeline

Chapter 5: Use Case Diagram

Level 0: VulCrypt

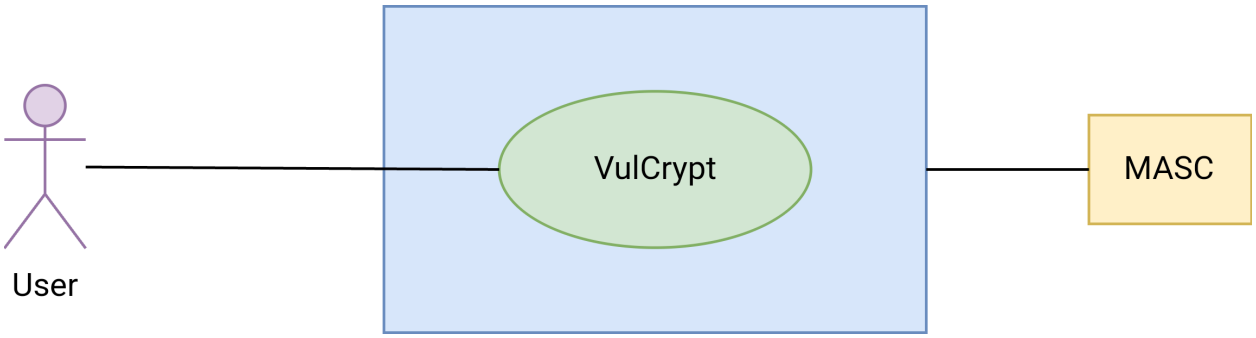


Figure 2: Level 0 Use Case

Primary Actor: User

Goal in the Context: The diagram represents the entire VulCrypt tool, showing an overview of the system's functionality and its interaction with the developer.

Level 1: Modules of VulCrypt

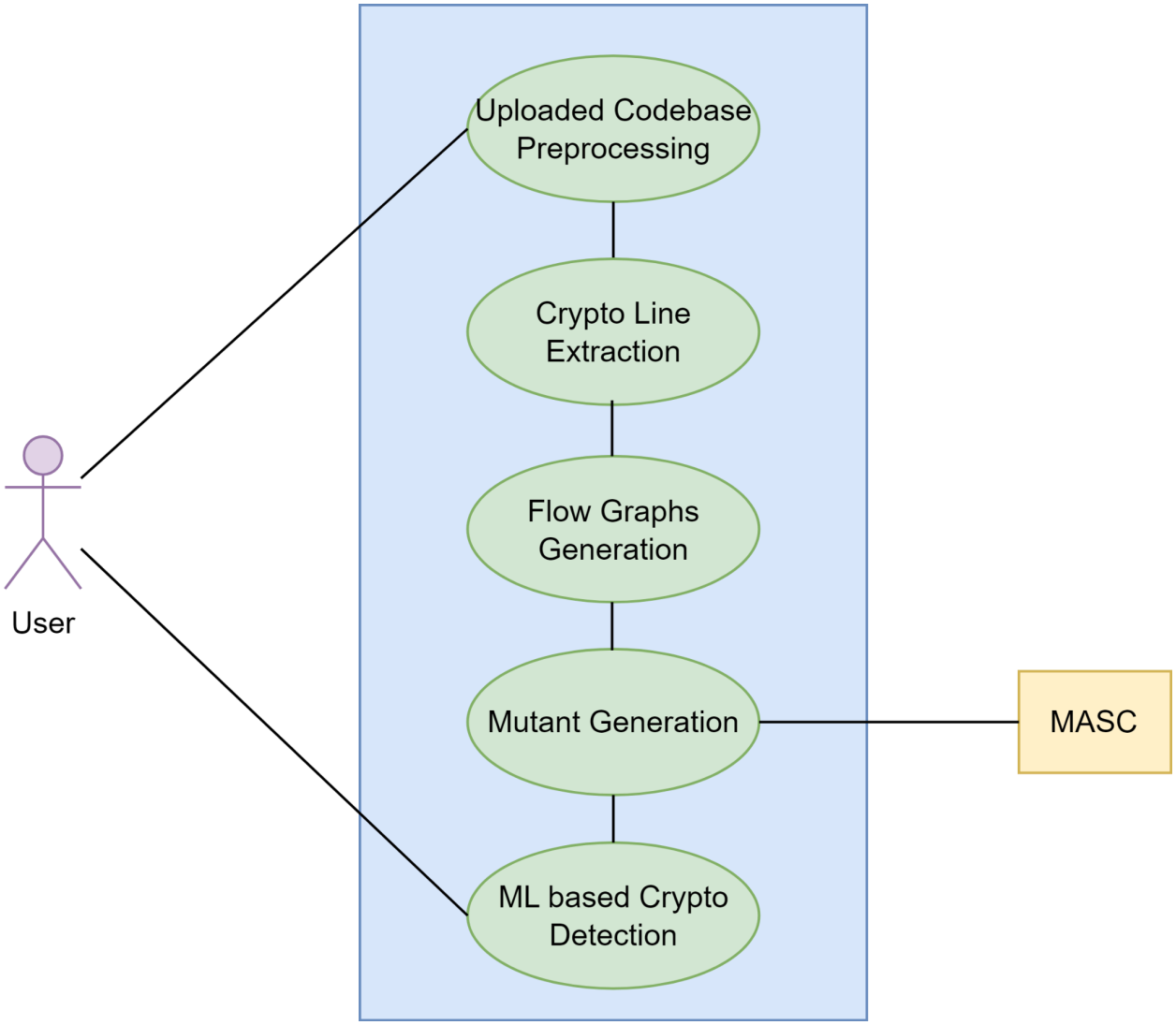


Figure 3: Level 1 Use Case

Primary Actor: User

Goal in the Context: The diagram shows all the major modules of VulCrypt. They are:

5.1.1 Uploaded Codebase Preprocessing: The developer provides the system with a zipped file of Java code or an APK. The module identifies and prepares the files for processing. Ensures the Java code is formatted uniformly, eliminating inconsistencies hindering subsequent analyses.

5.1.2 Cryptographic Line Extraction: Iterates over every Java file to search for cryptographic usages and extracts those line and isolate them for further analysis.

5.1.3 Flow Graphs Generation: Generates CFGs, DFGs, and ASTs for structural analysis, including control flows and method hierarchies. This is done to extract meaningful slices of Java code based on predefined criteria to focus on relevant sections for analysis. Flags variables and functions from the sliced code and normalizes their names such as VAR1, FUNC1 etc.

5.1.4 Crypto Analysis: Leverages MASC (Mutation Analysis for evaluating Static Crypto-API misuse detectors) to create mutants for training the machine learning models.

5.1.5 ML Based Crypto Detection: After learning from the MASC trained dataset, it predicts the normalized code slices if they’re vulnerable at all or not. Afterwards displays the analysis summary and detailed results in a dashboard.

Chapter 6: Class Based Diagram

This chapter describes the class-based modeling of the cryptographic API vulnerability detection tool.

6.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes-

- APKDecompiler
- CryptoLineExtractor
- CFGDFGExtractor
- CodeSlicer
- ASTGenerator
- CodeAbstraction
- MLDetector
- MASCMutantGenerator

6.2 Class Cards

Class cards for this tool are documented below-

APKDecompiler	
Attributes	**Methods**
- apkFilePath - decompiledCodePath	- decompile(apkFile) - handleExceptions() - saveDecompiledCode(destination)
Responsibilities	**Collaborators**

- Decompile APK files into Java source code. - Save decompiled source code for further processing.	- CryptoLineExtractor

CryptoLineExtractor	
Attributes	**Methods**
- cryptoLines - sourceCodePath	- extractCryptoLines(sourceCode) - identifyMisusePatterns(lines)
Responsibilities	**Collaborators**
- Extract cryptographic lines of code (e.g., API usage). - Identify potential cryptographic misuse patterns. - Flag lines for further analysis.	- CFGDFGExtractor

CFGDFGExtractor	
Attributes	**Methods**
- cfg - dfg - cryptoLines	- generateCFG(lines) - generateDFG(lines) - analyzeGraphs(graph)
Responsibilities	**Collaborators**
- Generate Control Flow Graphs (CFG) and Data Flow Graphs (DFG). - Analyze relationships and flow within the extracted code. - Provide graphs for slicing relevant code snippets.	- CodeSlicer

CodeSlicer	
Attributes	**Methods**
- codeSlices - cfg - dfg	- extractSlices(graph) - filterIrrelevantSlices(slices) - storeSlices(destination)
Responsibilities	**Collaborators**
- Extract meaningful code snippets based on predefined criteria. - Filter irrelevant code snippets. - Store code slices for further steps.	- ASTGenerator

ASTGenerator	
Attributes	**Methods**
- astRepresentation - codeSnippet	- generateAST(file) - analyzeAST(ast)
Responsibilities	**Collaborators**
- Generate Abstract Syntax Trees (ASTs) from code snippets. - Analyze ASTs for hierarchical structure and logic flow.	- CodeAbstraction

CodeAbstraction	
Attributes	**Methods**

- abstractedCode - ast	- simplifyAST(ast) - focusOnPatterns(abstractCode)
Responsibilities	**Collaborators**
- Simplify the ASTs into abstract code snippets. - Focus on essential patterns and ignore unimportant details. - Provide abstract code for training models.	- MLDetector

MASCMutantGenerator	
Attributes	**Methods**
- mutantFiles - inputCode	- generateMutants(file) - saveMutants(destination) - augmentTrainingDataset(mutants)
Responsibilities	**Collaborators**
- Augment datasets with mutants for ML-based detector training.	- MLDetector

MLDetector	
Attributes	**Methods**
- model - trainingData - predictions - detectionMetrics	- trainModel(data) - predict(file) - reportMetrics()
Responsibilities	**Collaborators**
- Train and apply ML models for analyzing and classifying code.	- MASCMutantGenerator

6.3 Class Responsibility and Collaboration Diagram

The CRC diagram for VulCrypt is given below-

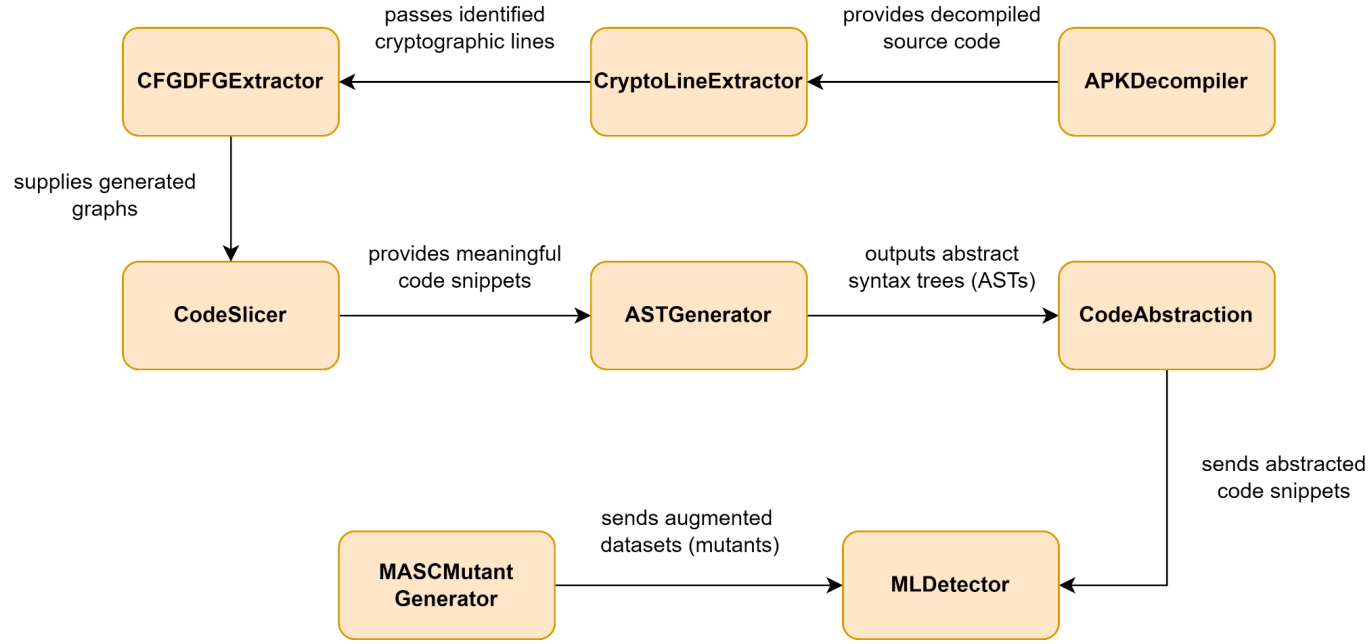


Figure 4: CRC Diagram

Chapter 7: Architectural Design

This chapter describes the architectural overview and architectural context diagram of VulCrypt.

7.1 Architecture Overview

VulCrypt will be a web application without the support of a database. Hence, it will need a 2-tier architecture. It will have a web user interface to get the user Inputs. The output will be processed on a remote server.

7.2 Architecture Context Diagram

An architectural context diagram is used to model how software interacts with entities external to its boundaries. Systems that interpolate with the target system can be represented as -

a. Subordinate Systems: Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.

b. Superordinate Systems: Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.

c. Peer-Level Systems: Those systems that interact on a peer-to-peer basis (i.e. information is either produced or consumed by the peers and the target system.

d. Actors: Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing. Each of these external entities communicates with the target system through an interface (the small shaded rectangles)

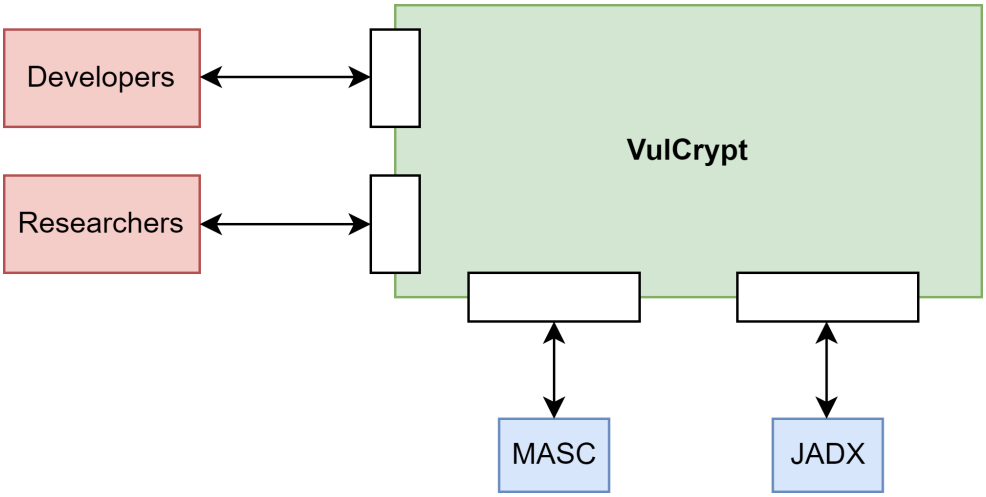


Figure 5: Architecture Context Diagram

7.3 Archetype Definition

The figure shows the archetype definition according to the tool's requirements analysis.

7.3.1 Client-Server Architecture:

Client-server architecture is a common architectural pattern in software systems where the responsibilities and functionalities of the system are divided into two distinct roles: the client and the server. These roles interact with each other to provide services, process requests, and exchange data.

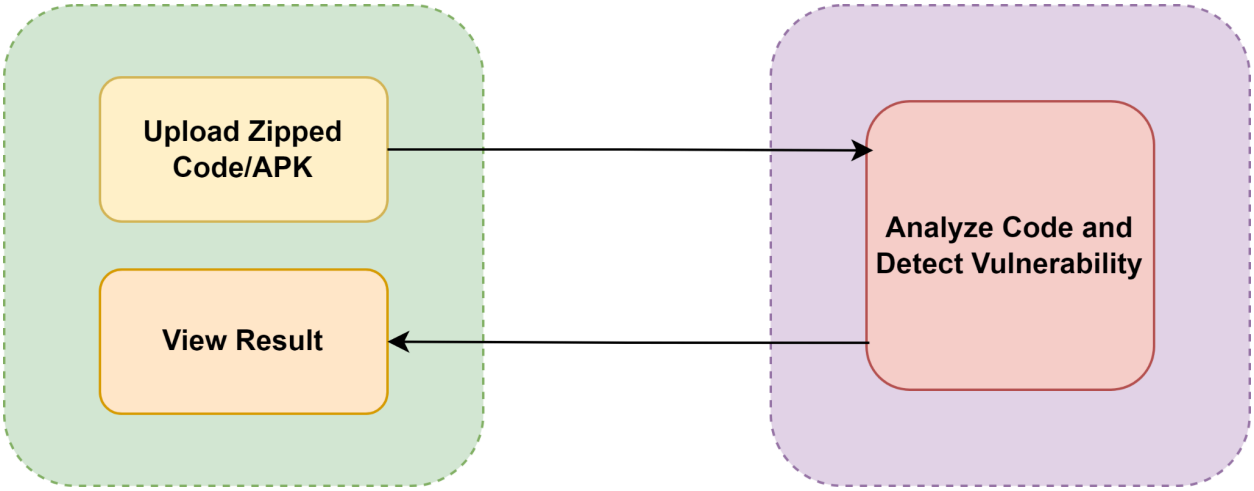


Figure 6: Client-Server Architecture

7.3.2 Pipeline and Filter Architecture:

Pipeline and Filter Architecture is a design pattern commonly used in software systems, particularly in data processing applications, to organize and streamline the flow of data through a series of processing stages or filters.

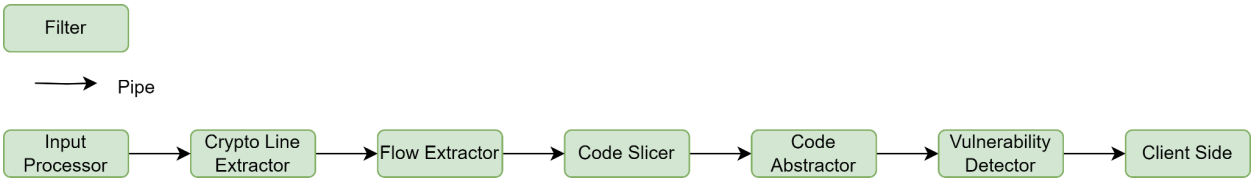


Figure 7: Pipeline and Filter Architecture

7.4 Refining Archetypes into Components

The figure below shows the refining into components based on the tool's archetype definition and requirements analysis.

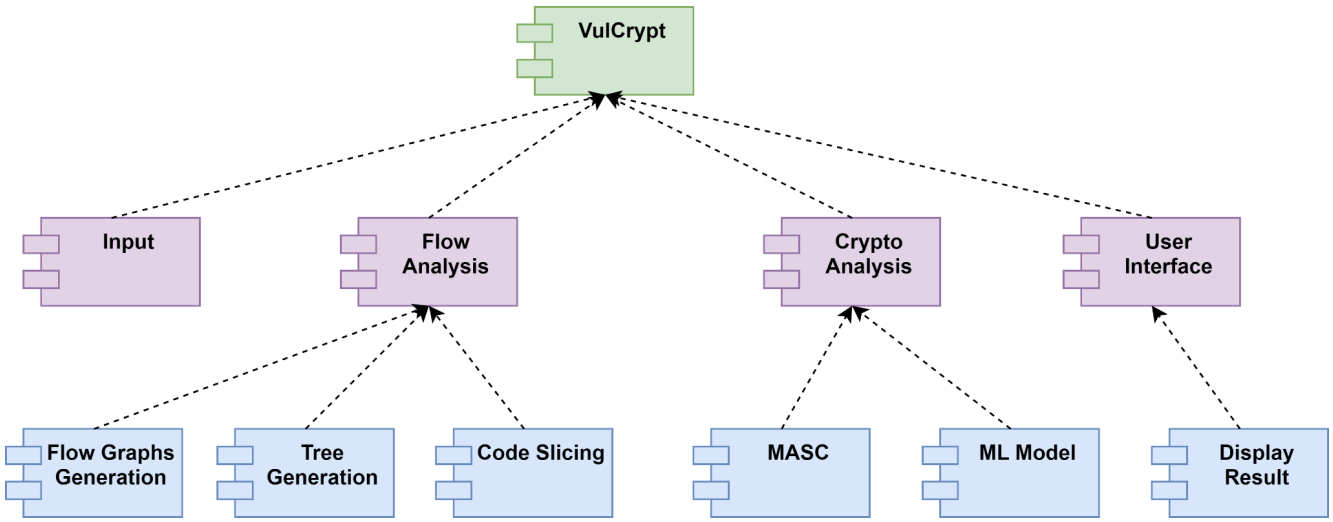


Figure 8: Refining Archetypes into Components

7.5 Elaborating the Deployment Diagram

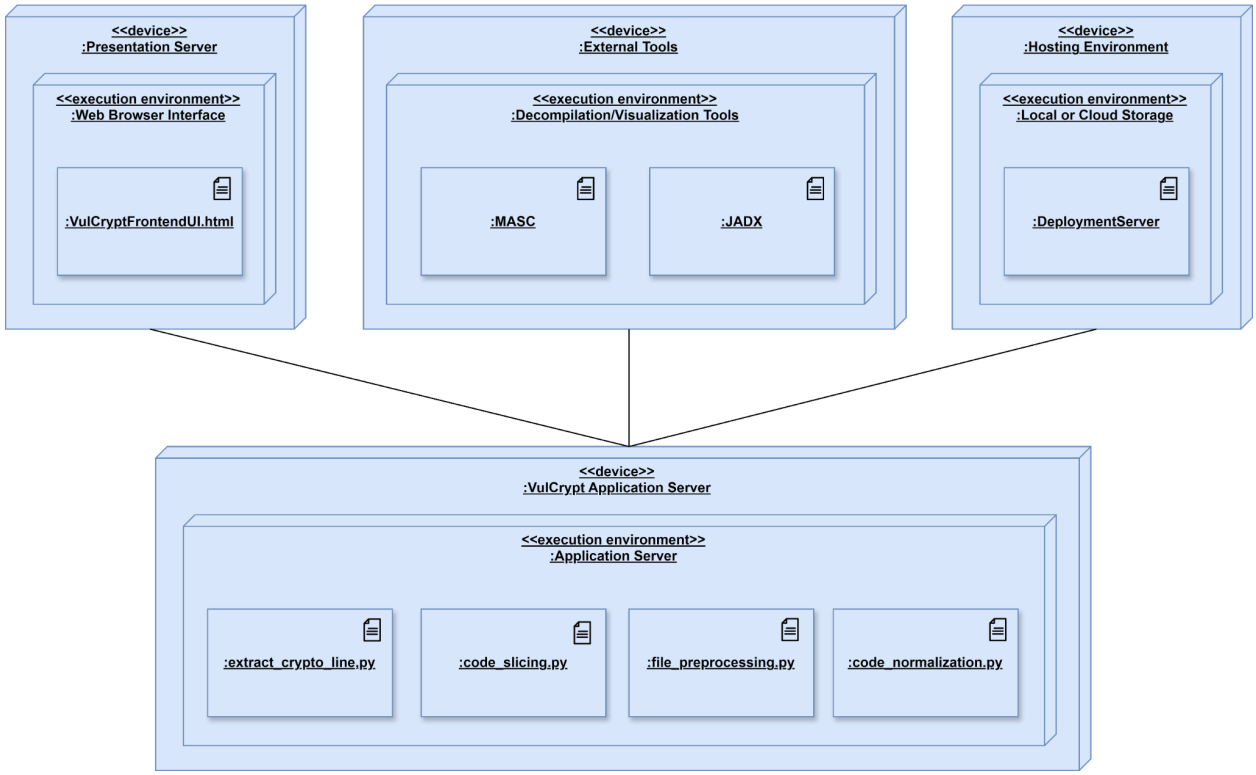


Figure 9: Deployment Diagram

Chapter 8: Primary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

8.1 High-Level Description of Testing Goals

Testing goals for this tool are represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure input is parsed properly
- To ensure that it analyzes System Complexity appropriately
- To ensure System Reconstruction
- To ensure that it produces valid and appropriate program slice
- To find any existing bugs
- To ensure that the tool runs smoothly

8.2 Test Cases

Test cases for the testing of this tool are documented below-

Test ID: T001

Test Case: Valid Input Processing

Test Scenario:

- Submit a valid Java file or APK file for analysis.
- Process the input through all stages of the pipeline.

Expected Outcome:

- VulCrypt accepts the file without errors.
- The analysis pipeline completes without interruptions.
- Results are accurately generated.

Pass/Fail: Pending

Test ID: T002

Test Case: Invalid Input Handling

Test Scenario:

- Submit an unsupported file type or corrupt file for analysis.

Expected Outcome:

- VulCrypt rejects the file with a clear and descriptive error message.

Pass/Fail: Pending

Test ID: T003

Test Case: Cryptographic Misuse Detection

Test Scenario:

- Analyze a Java file or APK containing known cryptographic misuse patterns.

Expected Outcome:

- VulCrypt correctly identifies and flags all instances of cryptographic misuse.
- No significant false positives or negatives are present.

Pass/Fail: Pending

Test ID: T004

Test Case: Machine Learning-based Detection

Test Scenario:

- Apply the ML-based detection model to an input file with extracted features.

Expected Outcome:

- The detection model classifies cryptographic usage as correct or incorrect with high accuracy.

Pass/Fail: Pending

Test ID: T005

Test Case: Results Display

Test Scenario:

- View the analysis results after processing an input file.

Expected Outcome:

- Results are displayed in a user-friendly format, such as tables or charts.
- Information is clear and interpretable.

Pass/Fail: Pending

Test ID: T006

Test Case: Proper Graphs Generation

Test Scenario:

- For a demo codebase, check if CFG/DFG/AST is produced accurately

Expected Outcome:

- Graphs are generated accurately.
- Information is clear and interpretable.

Pass/Fail: Pending

Test ID: T007

Test Case: Performance Under Load

Test Scenario:

- Process a large input file with multiple dependencies or extensive code.

Expected Outcome:

- VulCrypt completes the analysis within a reasonable time frame.
- The system remains responsive during the process.

Pass/Fail: Pending

Test ID: T008

Test Case: Error Handling in Pipeline Modules

Test Scenario:

- Introduce an error in one of the pipeline modules (e.g. during AST generation).

Expected Outcome:

- VulCrypt identifies and reports the error with a detailed message.
- Subsequent processing is halted gracefully.

Pass/Fail: Pending

Test ID: T009

Test Case: False Positive Testing

Test Scenario:

- Input a flawless zipped codebase or APK and see if it catches any vulnerability mistakenly

Expected Outcome:

- VulCrypt should not identify any vulnerability.
- The report should contain good remarks.

Pass/Fail: Pending

Chapter 9: Conclusion

In summary, VulCrypt represents a significant advancement in software engineering, leveraging cutting-edge research and a modular architecture. It empowers developers, architects, and researchers to harness Large Language Models for software system analysis and reconstruction. With a user-centric interface and a research-driven approach, it offers valuable insights into system complexity and improvements. The tool's adaptability positions it as a transformative asset in enhancing software quality and maintainability. Its success underscores the synergy between academia and practical software development, promising continued growth and impact in the field.

Reference

1. MASC: A Tool for Mutation-Based Evaluation of Static Crypto-API Misuse Detectors, https://www.adwaitnadkarni.com/downloads/ami-fse23-demo.pdf.
2. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques, https://www.adwaitnadkarni.com/downloads/ami-oakland22.pdf
3. Deep Learning Based Vulnerability Detection: Are We There Yet? https://ieeexplore.ieee.org/abstract/document/9448435.
4. VulCrypt: A Cryptographic API Misuse Detector, https://github.com/AbirAhmed72/VulCrypt